

COMP5047 Coursework

Subsystem: Society Leader App
Student Number: [19238600]

Task 2: Quality Requirements

Quality requirements define measurable standards for how a system performs its functions, specifying constraints on security, performance, reliability, and scalability rather than functional capabilities. This document specifies quality requirements for **FR-SL-1a:**

Activation of New Society, which enables society leaders to activate their approved societies and make them visible to students for membership.

The activation process involves authentication, notification delivery, database updates, and cross-subsystem data propagation, making it critical that the system operates securely, responds quickly, succeeds reliably, and scales to handle concurrent activations during peak registration periods. Each requirement includes a measurable metric, unit, precision, and target value, ensuring all requirements are testable and verifiable (Bass et al., 2021). These requirements apply to the Society Leader App subsystem operating within the University Student Unions (USU) platform serving 500+ universities.

Scenario 1: Society Leader Receives Approval Notification

Context: When a university student union officer approves a society application through the Student Union Management System, the Society Leader App must notify the designated society leader that their society is ready for activation. This notification enables the leader to know their society has been approved and can proceed with activation.

Quality Attribute	Requirement Statement	Metric	Unit	Precision	Target Value	How to Measure
-------------------	-----------------------	--------	------	-----------	--------------	----------------

Security	Notifications must only be delivered to the designated society leader. Other users cannot access or intercept activation notifications meant for different societies.	Unauthorized notification access	Attempts	1	0	Attempt to access notifications meant for other society leaders using different user accounts, verify system denies all unauthorized access.
Performance	Notification delivery must occur promptly after union approval to enable timely activation.	Notification delivery time	Seconds	0.1	≤ 30.0	Measure time from approval timestamp in Student Union Management System to notification receipt on leader's device.
Reliability	The system must successfully deliver notifications without failure even under varying network conditions.	Successful notification delivery rate	%	0.1	≥ 95.0	Send 100 test notifications under different network conditions (WiFi, 4G, 5G), count successful deliveries.
Scalability	The system must handle simultaneous notification delivery during peak approval periods at the start of semester.	Concurrent notifications processed	Count	1	≥ 200	Simulate 200 society approvals across different universities, verify all notifications delivered successfully.

Security in this context means preventing unauthorized users from accessing sensitive approval information, ensuring only the designated leader receives their notification.

Performance refers to how quickly notifications are delivered after union approval, enabling leaders to activate promptly during critical registration periods. **Reliability** means notifications are consistently delivered despite varying network conditions (mobile data, WiFi, email server load). **Scalability** ensures the system handles peak approval periods when hundreds of societies are approved simultaneously across multiple universities.

This scenario ensures society leaders are promptly informed when their societies are ready for activation. Push notifications with email fallback provide redundancy, while logging

enables traceability for debugging and compliance. All quality attributes are measurable and testable. Notification delivery time can be timed using timestamps, success rate tracked through monitoring, unauthorized access tested through penetration testing, and concurrent delivery stress-tested through load simulation.

Scenario 2: Society Leader Authenticates to Access Activation Feature

Context: Before a society leader can activate their approved society, they must authenticate their identity by logging into the Society Leader App. This ensures only authorized leaders can perform activation operations and maintains security of society management functions.

Quality Attribute	Requirement Statement	Metric	Unit	Precision	Target Value	How to Measure/Test
Security	Society leaders must authenticate using password plus multi-factor authentication. Failed login attempts must be logged and accounts locked after 3 consecutive failures for 15 minutes.	Unauthorized login attempts blocked	Attempts	1	0 successful	Attempt login with invalid credentials, stolen passwords, and bypassing MFA; verify all rejected, logged, and lockout activates.
Performance	The authentication process must complete quickly to avoid frustrating leaders wanting to activate.	Authentication response time	Seconds	0.1	≤ 3.0	Measure time from credential submission to authenticated session established, under normal load.

Reliability	The authentication system must correctly validate credentials every time without false rejections.	Authentication accuracy rate	%	0.1	≥99.0	Test 1000 valid login attempts, verify 99% are accepted; test 1000 invalid attempts, verify 99% are rejected.
Scalability	The system must support concurrent authentication requests during peak usage periods.	Concurrent authentications supported	Count	1	≥ 300	Simulate 300 simultaneous login attempts from different society leaders, verify all complete within performance target.

Security in this context means preventing unauthorized users from impersonating society leaders and gaining access to activation functions, using multi-layered authentication (password + MFA) and account lockout to defend against credential theft and brute-force attacks. **Performance** refers to how quickly the authentication process completes, ensuring leaders can quickly access activation features without frustrating delays. **Reliability** means the authentication system consistently and accurately validates credentials without false rejections (blocking legitimate users) or false acceptances (allowing unauthorized access). **Scalability** ensures the system handles concurrent authentication requests during peak periods when many leaders attempt to log in simultaneously.

This scenario ensures only authorized society leaders access activation functionality. Multi-factor authentication protects against password theft, account lockout prevents brute-force attacks, and comprehensive logging enables security monitoring and incident investigation. All quality attributes are measurable and testable. Authentication response time can be measured with automated tests, accuracy verified through positive and negative test cases, unauthorized access attempts tested through penetration testing, and concurrent authentication load-tested through simulation.

Scenario 3: Society Leader Performs Activation Operation

Context: After authentication, the society leader navigates to their approved society and taps the "Activate Society" button. The system must process this request by verifying authorization, updating the society status in the database, triggering notifications to relevant parties, and making the society visible to students in the USU Student App.

Quality Attribute	Requirement Statement	Metric	Unit	Precision	Target Value	How to Measure/Test
Security	Only the designated society leader for a specific society can activate that society. Activation attempts by other users must be denied and logged with full details (user ID, timestamp, IP address,).	Unauthorized activation attempts	Attempts	1	0 successful	Attempt activation using different society leader accounts, non-leader accounts, and expired sessions; verify all rejected and audit log contains complete information.
Performance	Activation processing must complete quickly with immediate UI feedback to confirm action received.	End-to-end activation time. Initial UI feedback time	Seconds Milliseconds	0.1 1	≤ 2.0 ≤ 500	Measure time from button tap to confirmation screen displayed. Measure time from tap to loading indicator appears.

Reliability	Activation operations must succeed consistently and data must be stored correctly without corruption or partial updates.	Activation success rate	%	0.1	≥ 99.5	Perform 1000 test activations under various conditions (network variability, concurrent operations), count successful completions with correct database state.
Scalability	The system must handle multiple society leaders activating their societies simultaneously during peak registration periods.	Concurrent activations processed	Count	1	≥ 50	Simulate 50 simultaneous activation requests from different society leaders, verify all complete successfully within performance targets.

Security in this context means enforcing role-based access control to ensure only authorized society leaders activate their specific societies, preventing fraudulent activities and maintaining system integrity through comprehensive audit logging. **Performance** refers to both immediate UI responsiveness (feedback within 500ms preventing user confusion) and complete transaction processing (activation within 2 seconds maintaining user satisfaction). **Reliability** means activation operations succeed consistently with data correctly stored through atomic database transactions, preventing partial updates or data corruption. **Scalability** ensures the system handles concurrent activation requests during peak periods without performance degradation or failures.

This scenario is the core function of FR-SL-1a. Immediate UI feedback prevents duplicate requests, fast processing maintains user satisfaction, atomic database transactions ensure data consistency, and concurrent handling supports start-of-semester peaks when hundreds of societies activate within days. All quality attributes are measurable and testable: response times measured with instrumentation, success rates tracked through monitoring, authorization verified through access control testing, and concurrent operations stress-tested through load simulation.

Scenario 4: Society Data Becomes Visible After Activation

Context: After successful activation, the society must immediately become visible and accessible to students in the USU Student App. Students should be able to search for the society, view its details, and subscribe for membership. This requires data propagation across subsystems and cache updates for real-time visibility.

Quality Attribute	Requirement Statement	Metric	Unit	Precision	Target Value	How to Measure/Test
Security	Activated society data must be visible only to authorized users; sensitive leader information must remain protected and not exposed to students.	Unauthorized data access	Attempts	1	0 successful	Attempt to access society data from students at different universities, verify access denied; verify leader contact details not exposed in student-facing APIs.
Performance	Society data must propagate to USU Student App and become searchable promptly after activation.	Data propagation time	Seconds	0.1	≤ 10.0	Activate society, immediately search for it in the USU Student App, measure time until society appears in search results.
Reliability	Activated societies must be consistently visible to all eligible students without gaps or inconsistencies.	Data consistency rate	%	0.1	99.0	Activate 100 societies, verify at least 99 are immediately visible to 10 test student accounts at the same university, check data matches activation details.

Scalability	The system must support simultaneous data propagation for multiple society activations without bottlenecks.	Concurrent propagations handled	Count	1	≥ 50	Activate 50 societies simultaneously, verify all become visible to students within performance target, check for data corruption or missing societies.
--------------------	---	---------------------------------	-------	---	-----------	--

Security in this context means enforcing access control boundaries between universities, ensuring students only see societies from their own university, and protecting society leader personal information from public exposure through careful API design. **Performance** refers to how quickly activated society data propagates across subsystems and becomes searchable in the USU Student App, enabling immediate student discovery and membership. **Reliability** means data is consistently visible to all eligible students without gaps, partial updates, or inconsistencies caused by caching issues or race conditions. **Scalability** ensures the system handles concurrent data propagation operations during peak activation periods without bottlenecks or delays.

This scenario completes the activation workflow by ensuring students can immediately discover and join activated societies. Data propagation across subsystems (Society Leader App → backend services → USU Student App) must be fast and reliable, cache invalidation ensures real-time visibility, and access control protects privacy. All quality attributes are measurable and testable: propagation time measured by searching immediately after activation, consistency verified by checking multiple student accounts, unauthorized access tested through cross-university access attempts, and concurrent propagation stress-tested through bulk activation simulation.

Task 3: Use Case and Activity Modelling for FR-SL-1a

FR-SL-1a “Activation of New Society” describes how a society leader activates a union-approved society so it becomes visible to students for membership. The use case diagram shows the Society Leader as the primary actor interacting with the Society Leader App subsystem, with the Student Union Management System and USU Student App as supporting systems for approval status and visibility.

The activity diagram details the main flow: leader logs in, selects an approved society, reviews details, and chooses “Activate Society”. The system verifies approval and authorization, updates the society status, triggers notifications, and starts data propagation to the USU Student App. Alternative paths capture invalid credentials, missing approval, or backend failures, ensuring the model covers both successful and exceptional activation behaviour.

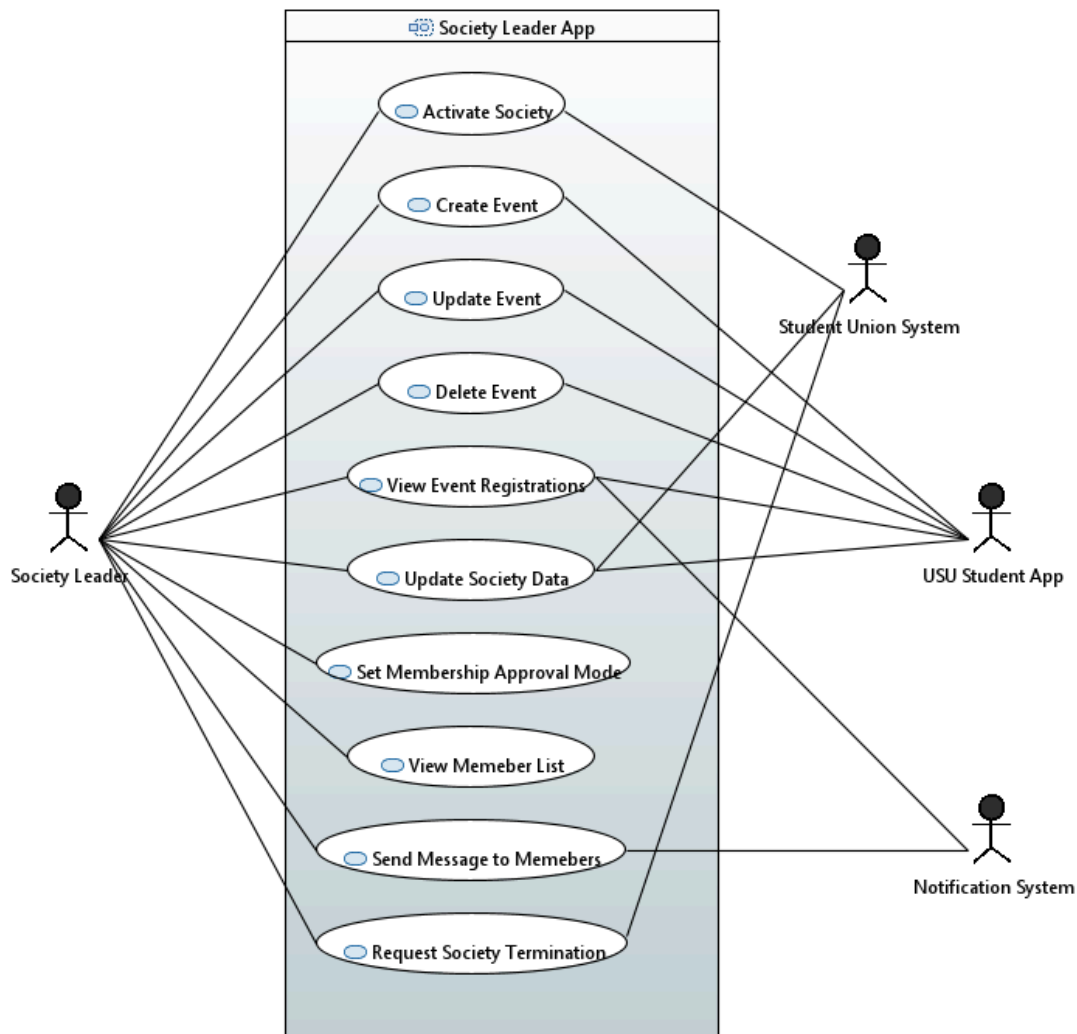


Figure 1: UML Use Case Diagram

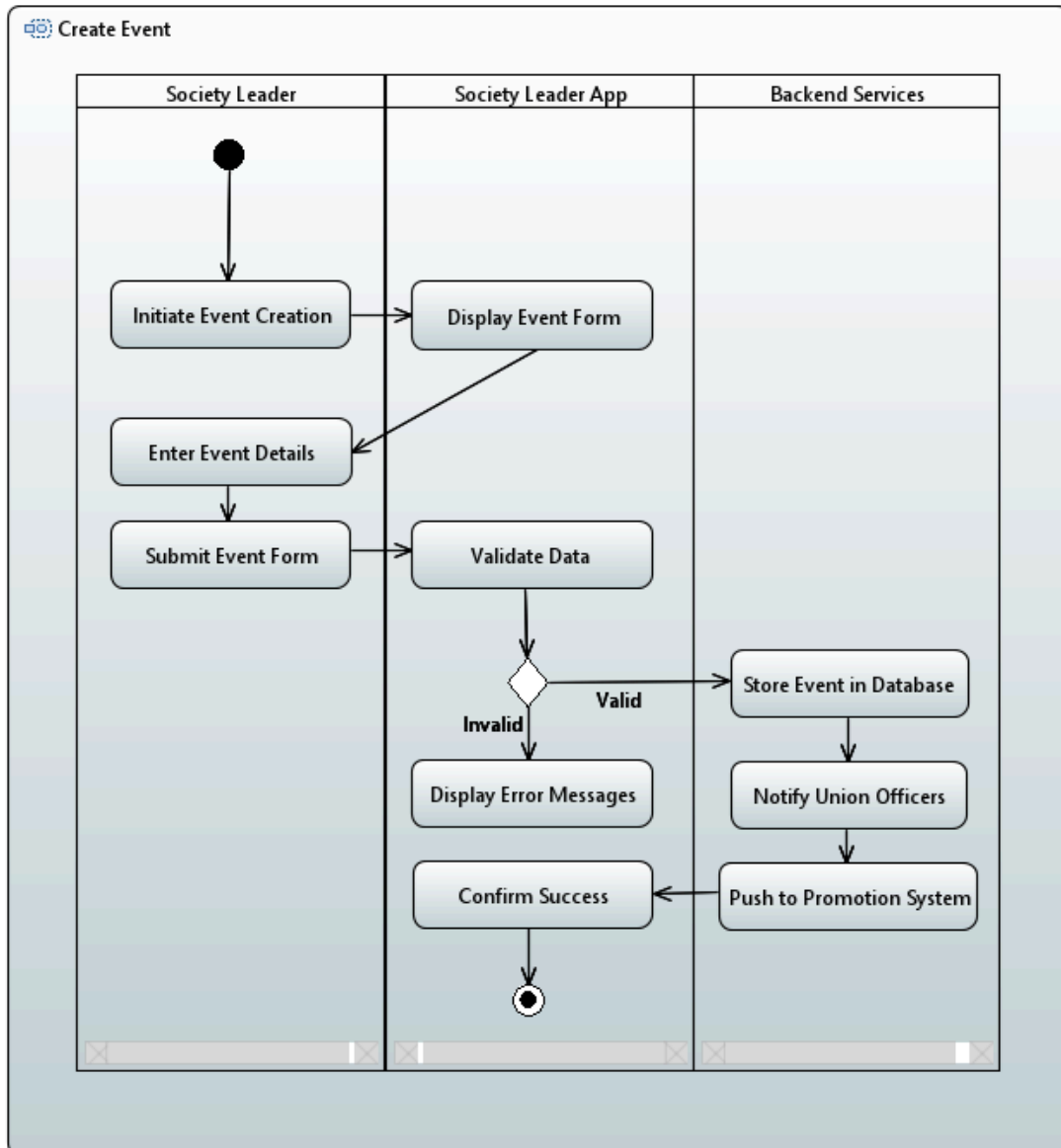


Figure 2: UML Activity Diagram

Task 4a: Subsystem Architectural Design – Society Leader App

The Society Leader App subsystem is implemented as a set of backend services: Society Management Service, Event Management Service, Membership Management Service, and Notification Orchestrator Service. The component diagram shows how these services collaborate and interact with external subsystems: the Student Union Management System via UnionApprovalAPI for approval status, the USU Student App backend via StudentAppSyncAPI, StudentAppEventsAPI, and StudentAppMembershipAPI to expose societies, events and memberships to students, the Generic Notification Gateway via NotificationGatewayAPI for outbound email/SMS/push, and the Shared Identity Provider via IdentityAPI for authentication. Cross-subsystem calls from the Society Leader App to the USU Student App are modelled as asynchronous interactions to support scalability during peak activation periods while still satisfying the security and reliability requirements defined for FR-SL-1a.

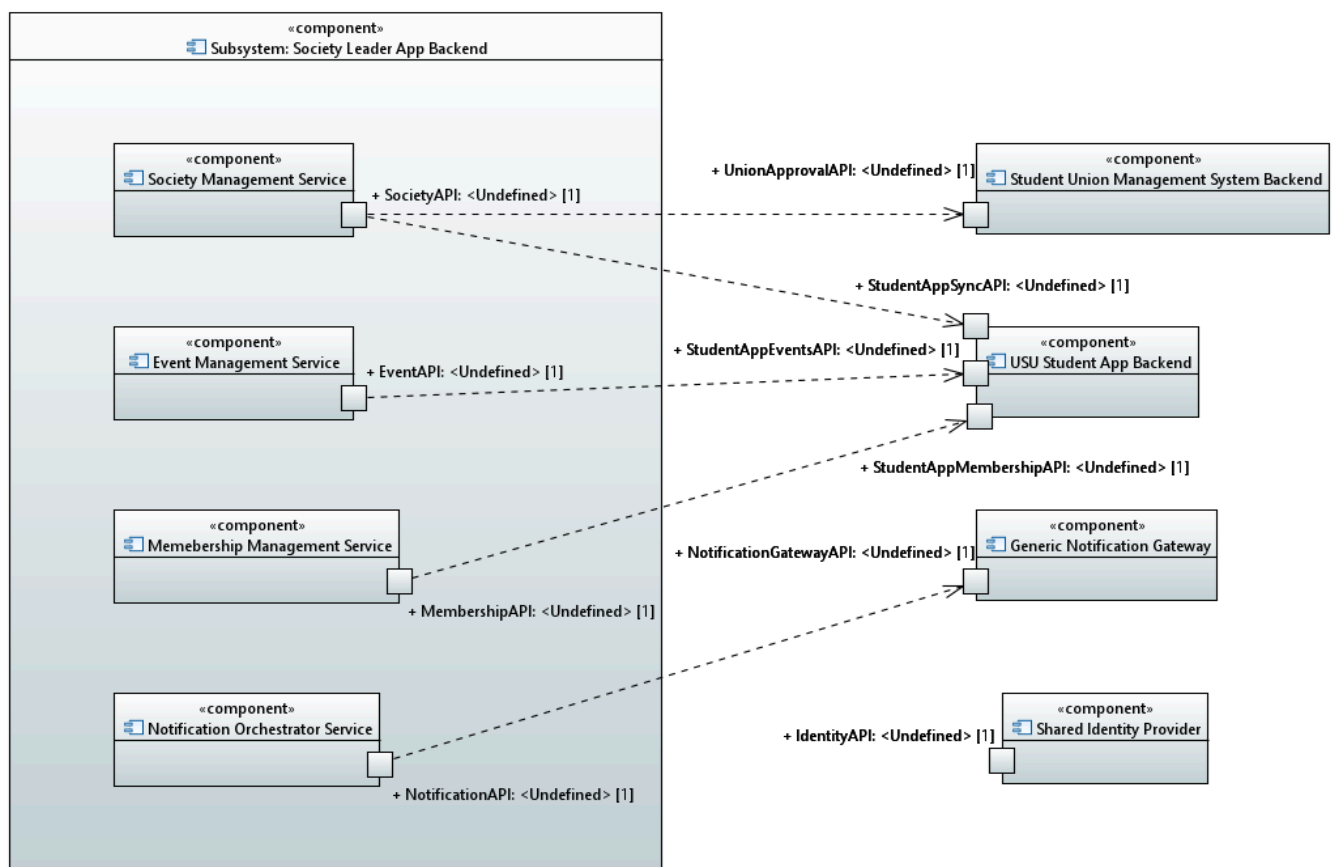


Figure 3: UML Component Diagram

Task 5(a): Class Diagram for Event Management Service

The class diagram models the internal structure of the Event Management Service responsible for creating and maintaining society events. EventService acts as the application façade providing operations such as createEvent, updateEvent, deleteEvent, getEvent, listEventsForSociety, and listRegistrations, and it coordinates persistence and notifications. Event represents the core event entity with attributes for society, name, description, venue, start/end time and status, and is associated one-to-many with EventRegistration, which records individual student registrations, timestamps, and registration status. EventRepository and RegistrationRepository encapsulate database access for events and registrations respectively, while NotificationClient abstracts calls to the Notification Orchestrator so that EventService can trigger event-created, updated, or cancelled notifications without depending on a particular messaging technology.

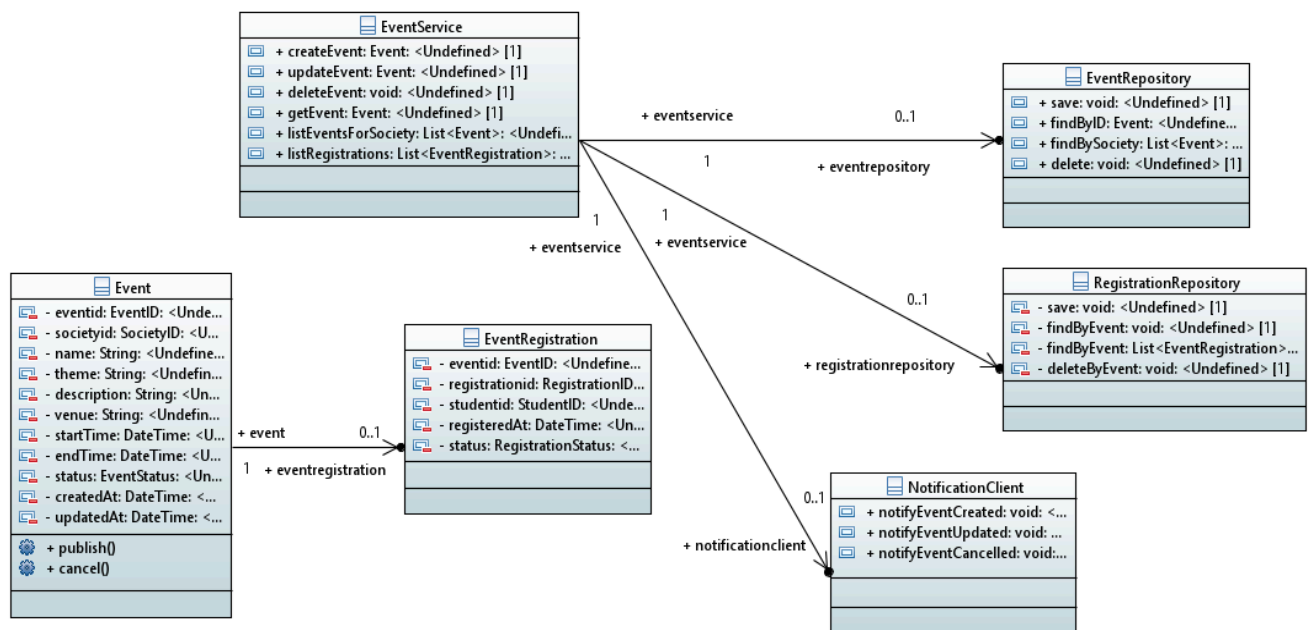


Figure 4: UML Class Diagram

Task 5(b): createEvent Sequence Diagram

The createEvent sequence diagram shows the dynamic behaviour when a society leader submits the event creation form. The SocietyLeaderUI sends createEvent(eventFormData) to EventService, which first validates the form data and, on validation failure, returns a validationError(messages) response to the UI to inform the leader of all issues. In the successful path, EventService persists the new Event via EventRepository.save(event) and, once the save completes, calls NotificationClient.notifyEventCreated(event) to trigger downstream notifications before returning eventCreatedConfirmation(eventId, successMessage) to the UI; this ensures the operation covers data validation, atomic persistence and user feedback consistent with the responsibilities modelled in the class diagram.

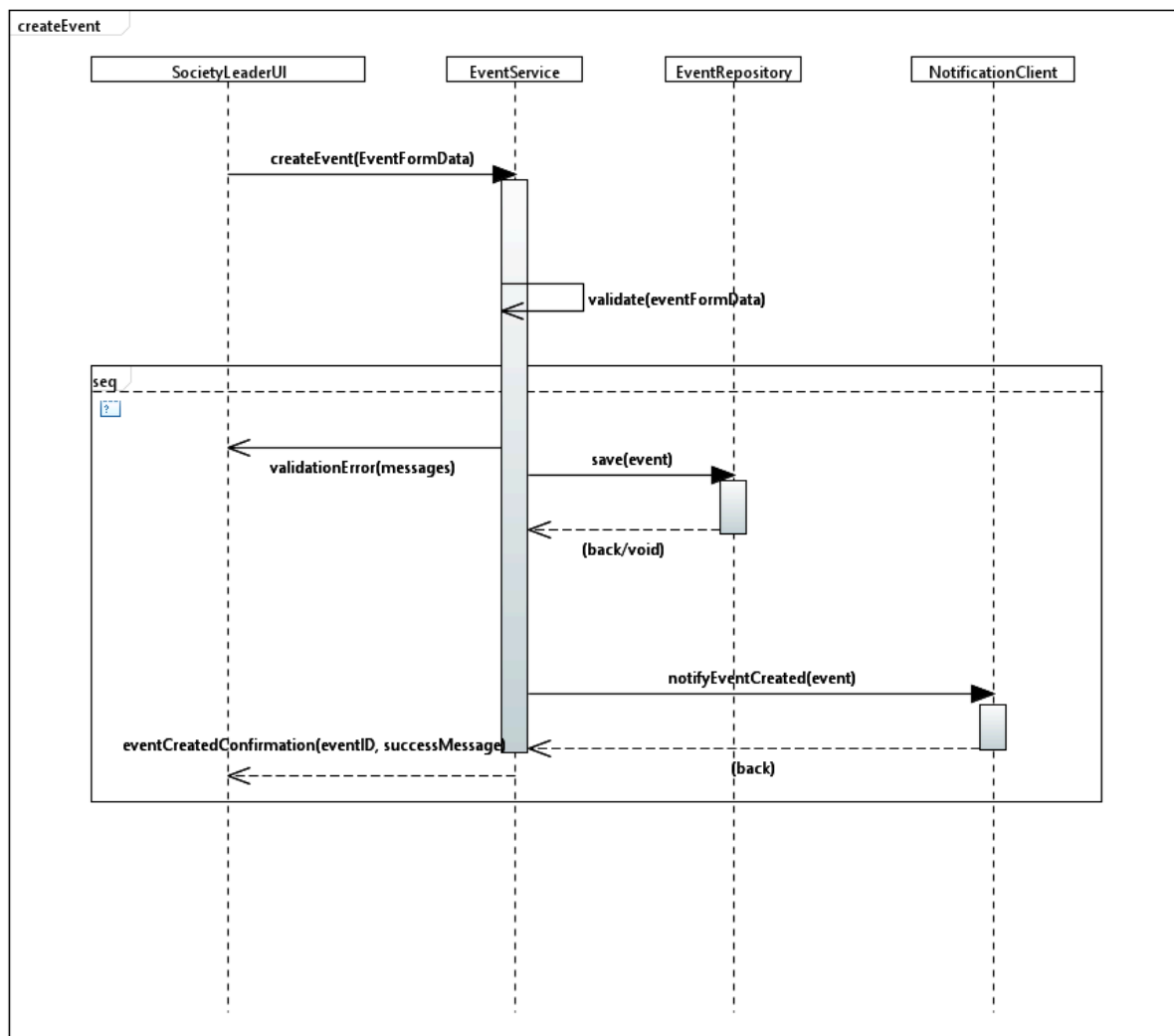


Figure 5: UML Sequence Diagram

Individual Teamwork Report

In this coursework I was responsible for the Society Leader App subsystem, including the quality requirements for FR-SL-1a, the use case and activity models, the subsystem component diagram, and the Event Management Service class and sequence diagrams.

The team organised work through a shared GitHub repository and regular online meetings, where we agreed the system-level APIs and checked that each subsystem design remained compatible with the overall USU architecture described in the group report. During the semester I shared interim versions of my diagrams with the group for feedback and used their comments to align naming, responsibilities and interfaces, particularly around the activation workflow and cross-subsystem synchronisation. Overall the collaboration was effective and the final models for the four subsystems form a consistent view of the platform.

References

Bass, L., Clements, P. and Kazman, R. (2021) *Software Architecture in Practice*. 4th edn.

Upper Saddle River: Addison-Wesley. Available at:

<https://www.oreilly.com/library/view/software-architecture-in/9780136885979/> (Accessed: 9th November 2025).